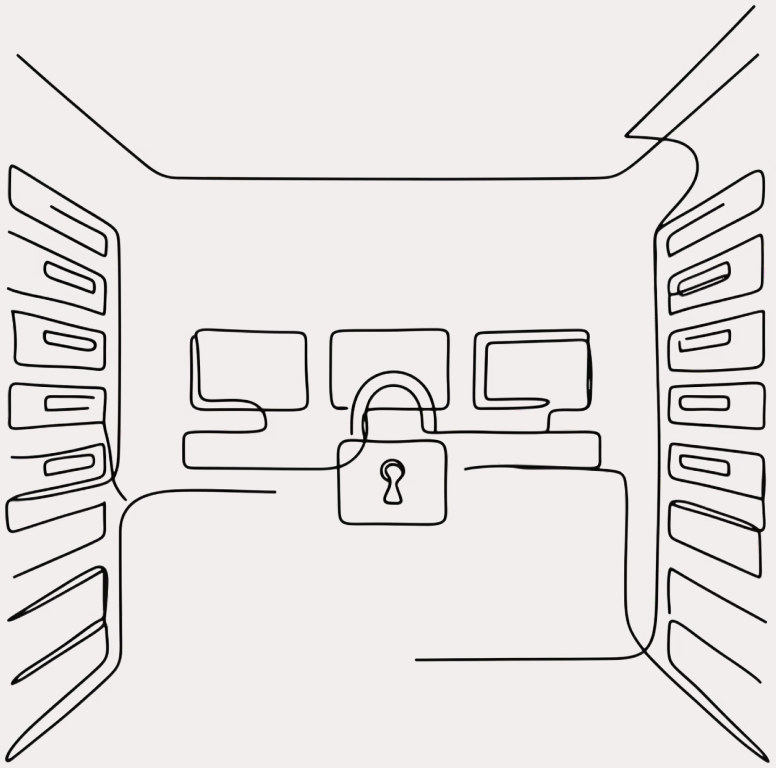


Vulnerabilidades Críticas en PHP: PHP2Shell

Análisis, Impacto y Mitigación Urgente

MASTERCLASS · 60 MINUTOS TEÓRICO-PRÁCTICO

Requisitos previos: conocimientos básicos de desarrollo web y lógica de servidores PHP.

unaWeb

Bloque 1: ¿Qué es PHP2Shell y por qué es crítico?

El panorama actual

PHP impulsa gran parte de la web, incluyendo CMS masivos como WordPress, convirtiéndolo en uno de los blancos favoritos de atacantes automatizados.

Definición de PHP2Shell

No es una función nativa, sino un término que engloba fallos críticos donde un atacante transforma una brecha en la lógica PHP en una **consola de comandos interactiva (Shell)** dentro del servidor operativo.

Impacto empresarial

- Pérdida total de confidencialidad
- Manipulación de bases de datos
- Inyección de malware y web shells persistentes
- Exfiltración masiva de datos corporativos



Bloque 2: Anatomía de la Vulnerabilidad

Tres mecanismos técnicos principales permiten que un entorno PHP derive en un ataque de tipo Shell:

🔓 Falta de Sanitización (Input Validation)

Parámetros GET/POST o cabeceras HTTP manipuladas que se pasan directamente a funciones peligrosas como `eval()`, `system()`, `exec()`, `passthru()` o `shell_exec()`.

⚠️ Deserialización Insegura (unserialize)

Si la aplicación procesa objetos arbitrarios del usuario sin controles estrictos, los atacantes pueden ejecutar código arbitrario durante la carga de dichos objetos — conocido como **PHP Object Injection**.

📂 Inclusión de Archivos LFI/RFI

Permitir que rutas controladas por el usuario carguen archivos externos que luego son interpretados como código fuente ejecutable por el servidor.

⚠ Un atacante que inyecta `8.8.8.8; cat /etc/passwd` en un parámetro no sanitizado ejecuta comandos con los privilegios del usuario web.

Bloque 3: Taller — La IA como Auditor de Código

Escenario: analizar un fragmento de código PHP vulnerable usando IA como herramienta de auditoría automatizada.

Paso A · Código Vulnerable (5 min)

```
$resultado
```

"; } ?>

Paso B · Prompt de Auditoría IA (10 min)

Instruir a la IA (ChatGPT, Claude, Deepseek) con rol de **Auditor de Ciberseguridad Senior** para que identifique la línea vulnerable, explique el vector de explotación y entregue el código corregido.

Paso C · Análisis de la Respuesta (5 min)

La IA identifica de inmediato el peligro de `shell_exec()` concatenado con entradas del usuario y sugiere alternativas seguras:

- Usar `escapeshellarg()` para escapar la entrada
- Validar con `filter_var($ip, FILTER_VALIDATE_IP)` antes de cualquier ejecución
- Aplicar listas blancas estrictas de valores permitidos

La IA reduce de horas a segundos el tiempo de detección de vulnerabilidades RCE en código PHP.

Bloque 4: Mitigación Urgente y Hardening

Desactivar funciones peligrosas en php.ini

```
disable_functions = exec, passthru, shell_exec,
system, proc_open, popen
```

Principio de menor privilegio

El usuario del servidor web (`www-data` / `nginx`) debe tener permisos mínimos de lectura/escritura únicamente en los directorios indispensables.

Desarrollo seguro

Validar, filtrar y sanitizar siempre cualquier dato externo (POST, GET, Cookies). Evitar el uso dinámico de `eval()` o ejecución de cadenas de texto.

Monitoreo y detección de anomalías

Implementar **Web Application Firewalls (WAF)** y análisis automatizado de logs en busca de patrones de escape (`;`, `&&`, `|`) en peticiones HTTP inusuales.

